

Agentic AI and Code Reviews

Patterns and Case Studies

Michelle Gill (Engineered By AI)

Randy Shoup (CircleCI)

AI-Assisted Code Accelerating

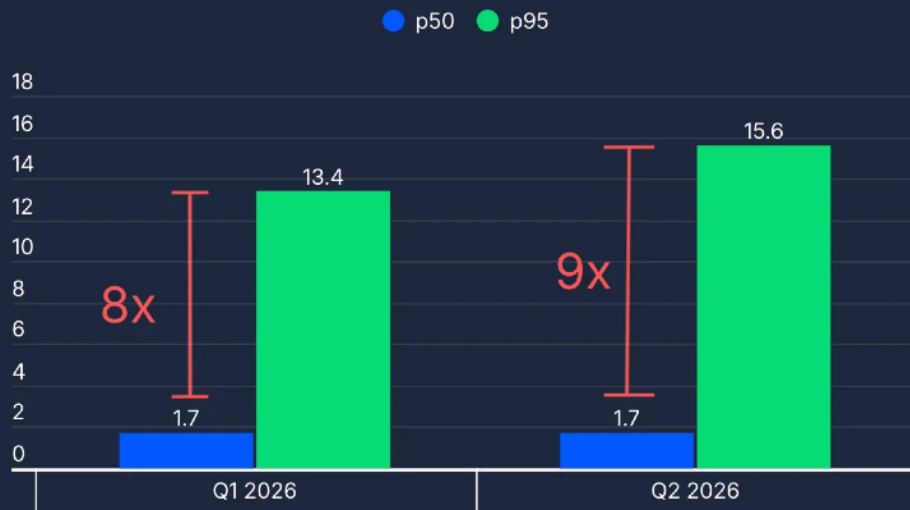
■ Percentage of code that is AI-authored



Best Teams Getting Faster

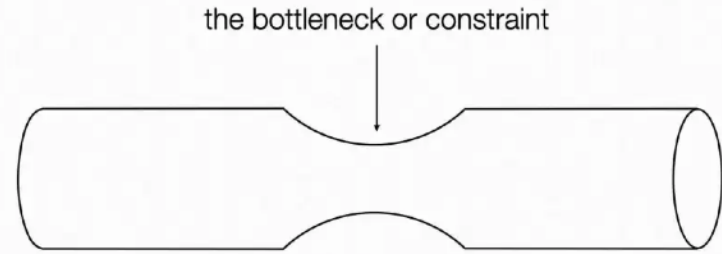


The growing gap between median and p95 throughput on main



Theory of Constraints

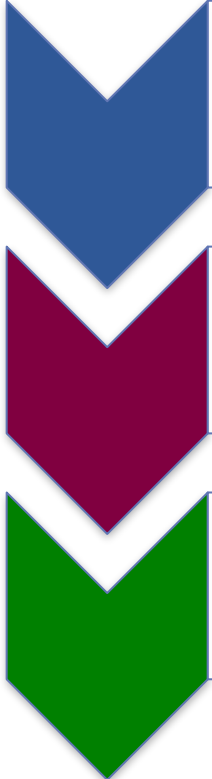
- Every system has at least one constraint
- Focusing improvement efforts on that constraint is the only way to improve throughput of the system
- Any improvement not at the bottleneck is an illusion



“AI-driven changes are breaking more often and taking teams longer to fix, making validation and integration the primary bottleneck.”

-- CircleCI 2026 State of Software Delivery Report

AI and Code Reviews

- 
- Code Review as Constraint
 - Patterns of Review
 - Case Studies

AI and Code Reviews



- Code Review as Constraint



- Patterns of Review



- Case Studies

Should We Remove Code Reviews?

The value of code review goes beyond identifying functional issues:

- Non-functional requirements (readability, maintainability)
- Knowledge sharing
- Identifying alternate solutions
- Mitigating risk before it reaches production

How do we shift to AI-native development without losing quality or trust?

What Is Code Review?

Examination of source code by an entity that is **not the author**, with the intention of making a judgment that **leads to an action**

Removing Humans from the Critical Path



1. Push as much as possible into deterministic checks
2. Risk-tier the degree of human involvement
3. Product durable and inspectable records of every decision

AI and Code Reviews



- Code Review as Constraint



- Patterns of Review

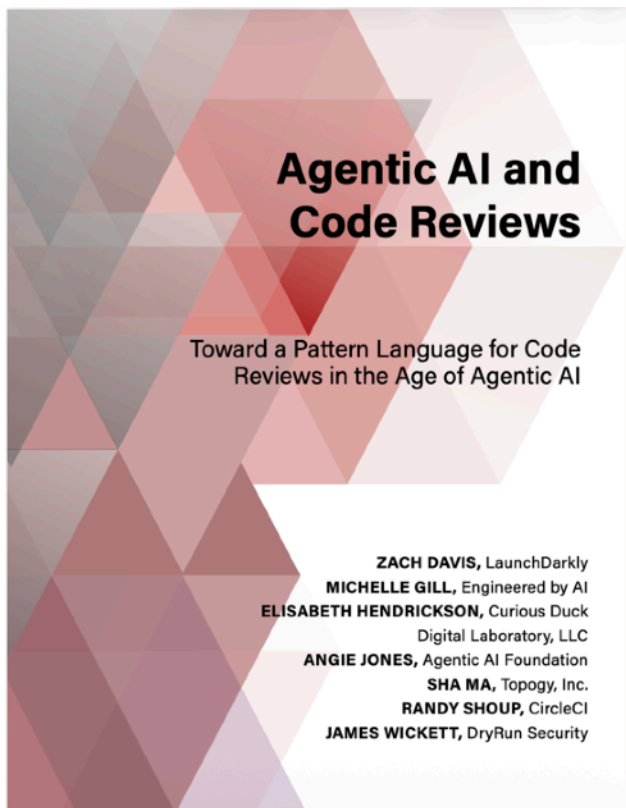


- Case Studies

With human authorship and human review, we were able to get away with less rigor.

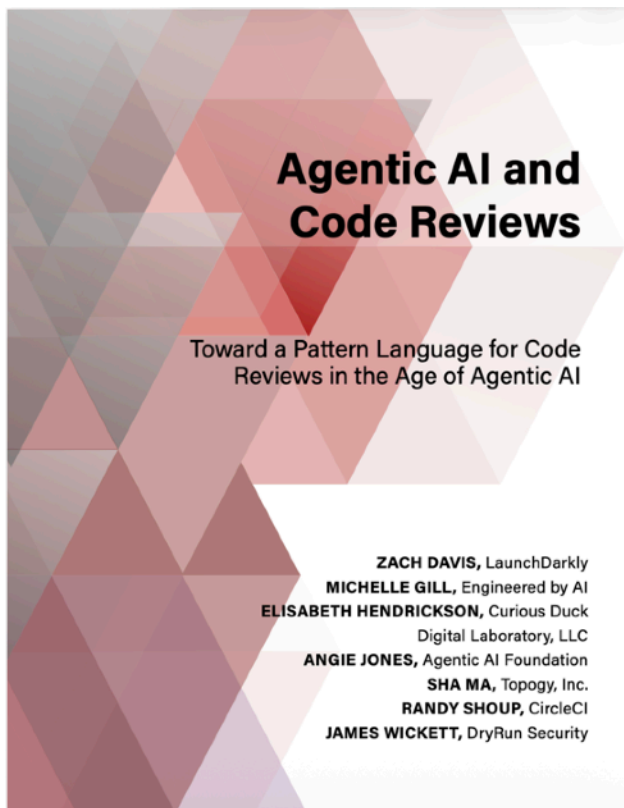
With machine authorship and machine review, **we have no choice but to systematize the rigor.**

Pattern: Small Incremental Changes



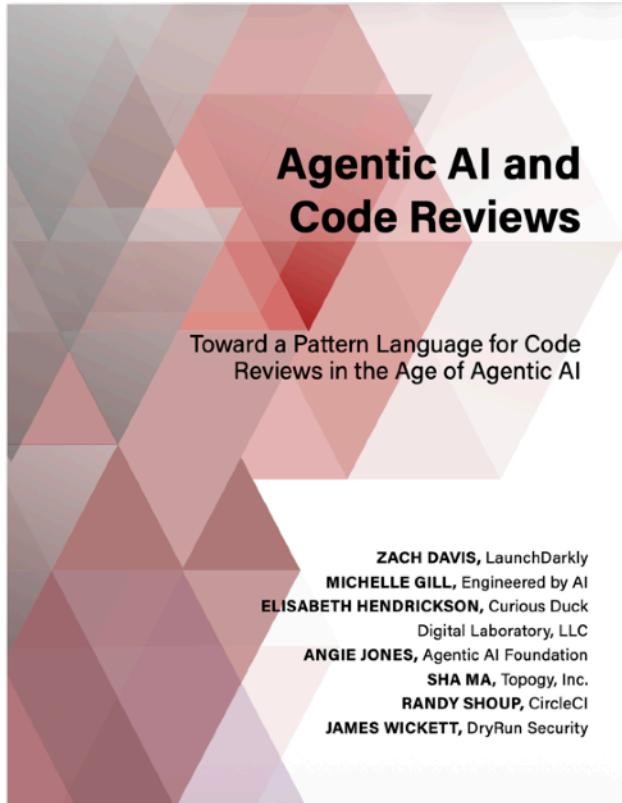
- Easier to understand, to roll out, to roll back
- Isolate functional changes from migrations and dependency updates
- Smaller context window is more token-efficient
- By default, LLMs make large changes worse

Pattern: Shift Left



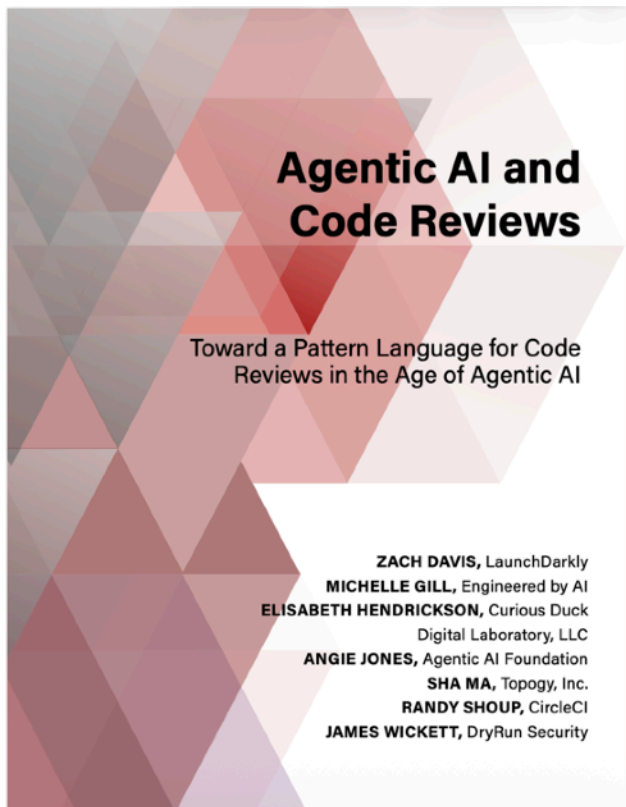
- Move everything possible into the Inner Loop for faster real-time feedback
- Deterministic checks via static code analysis, linting, standards checking
- Repository-specific rules, constraints, conventions

Pattern: Design for Verification



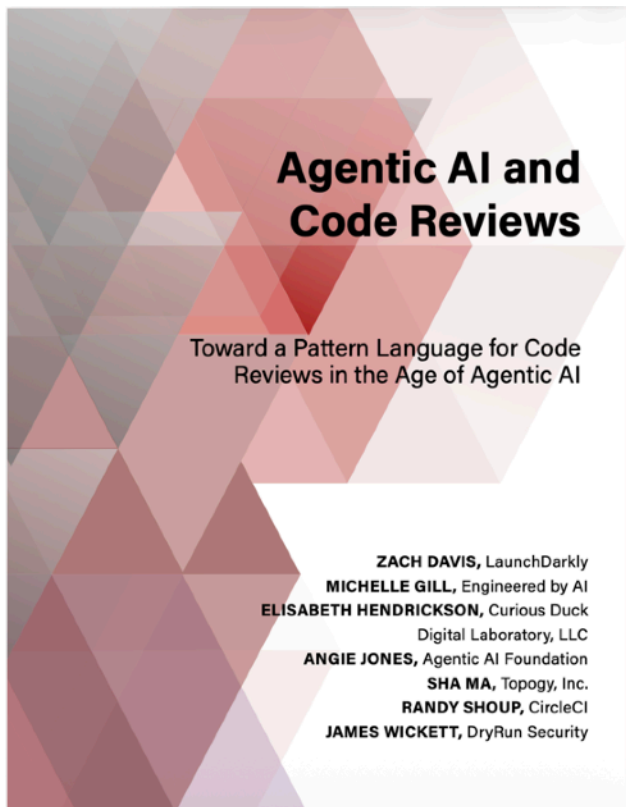
- BDD / ATDD to explicitly specify intent and behavior
- Service / API expectations as contract tests and schema validations
- Focus on test case reliability
- Useful context and fast feedback for agents

Pattern: Deterministic Engineering



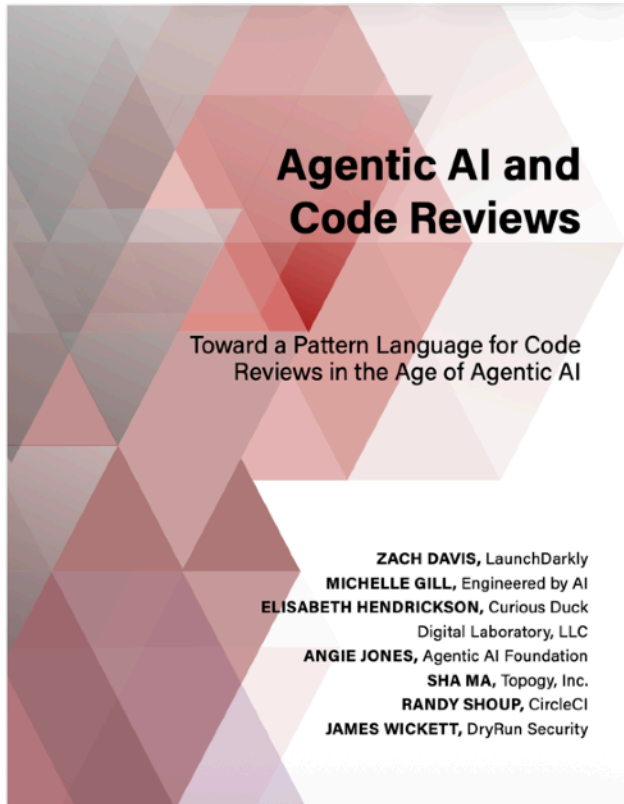
- Compiled languages
- Strong typing
- Explicit dependencies
- Idempotent operations
- Observable state transitions

Pattern: Spec-Driven Development



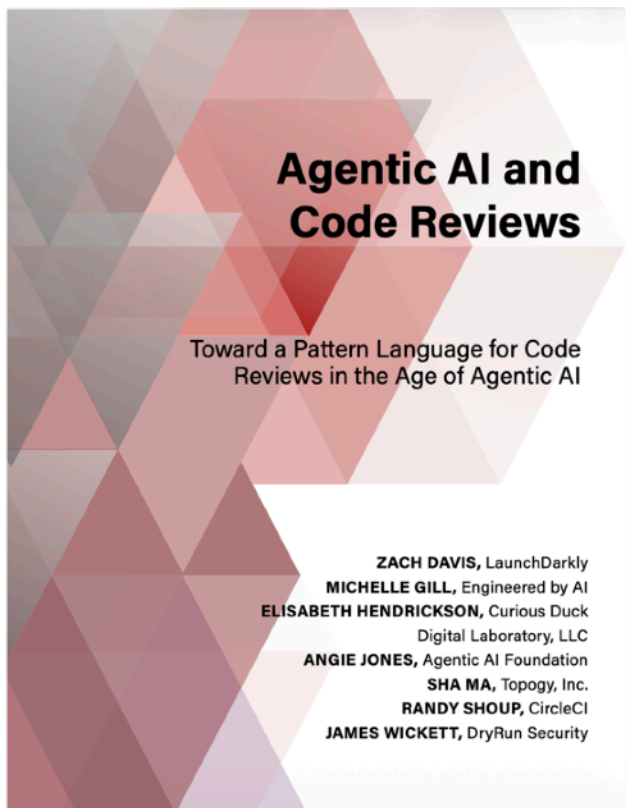
- Specify behavior in a structured specification
- Review and iterate on intent before implementing
- Ideally all bugs are spec violations
- Use spec to evaluate correctness and compliance

Pattern: Context Engineering



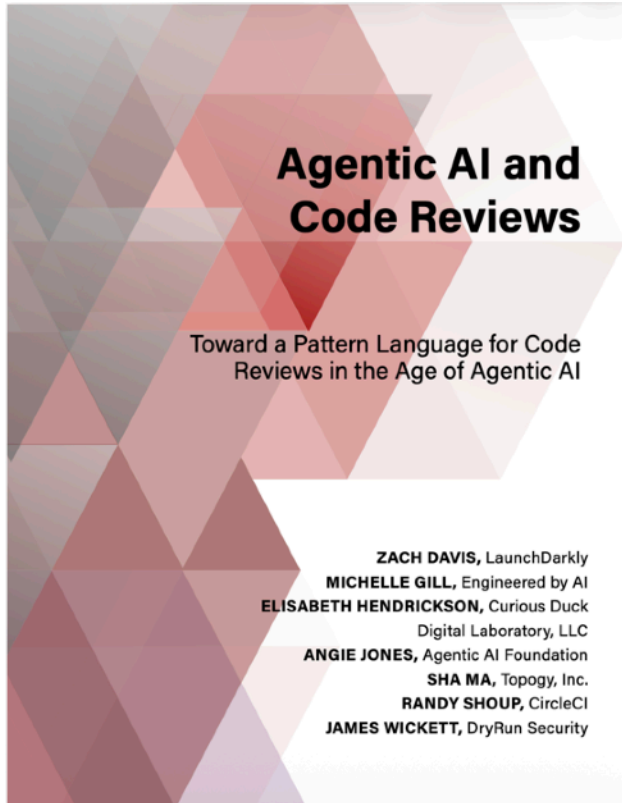
- Knowledge Graph with documentation, system behavior, code reviews, incidents, past decisions, etc.
- Structured repository metadata and standard AGENTS.md files
- API contracts, architectural constraints and service boundaries
- Security and compliance policies

Pattern: Continuous Code Quality



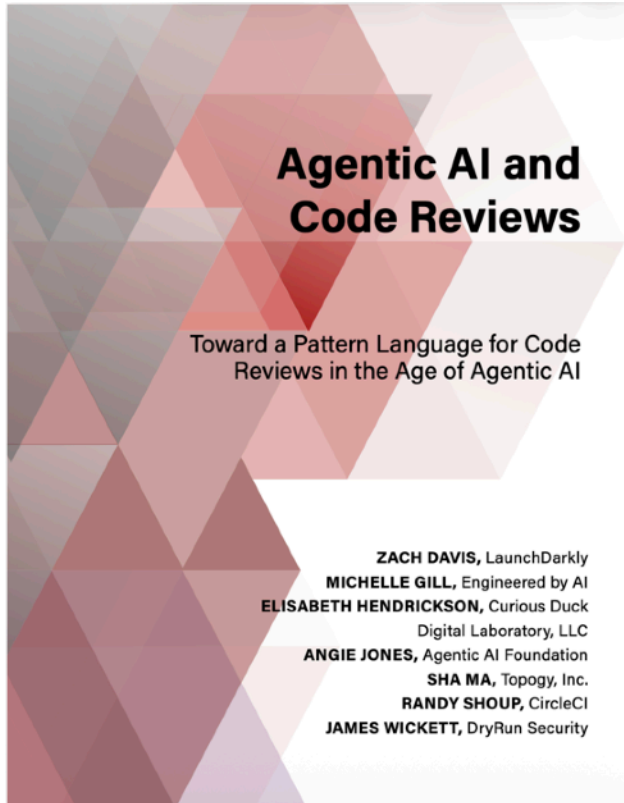
- Review agent provides fast feedback in the Inner Loop of code generation
- Inside IDE, in pre-commit hooks, in pre-merge hooks
- Engineers configure and tune review agent

Pattern: Adversarial Review



- Separate generation and evaluation with independent review agent (“LLM as Judge”)
- Cross-session or Cross-model
- Adversarial prompting

Pattern: Multi-Agent Review



- Independent specialist review agents for independent concerns
- Triage agent
- Correctness, performance, security, compliance, etc.
- Ensemble coordination

AI and Code Reviews



- Code Review as Constraint



- Patterns of Review



- Case Studies

Encode What We've Already Learned

Built with AI from day one, **DryRun Security** hit a bug that let customer data cross tenant boundaries. Missed by reviewer.

DryRun now reviews every PR against contextual security logic, returns useful feedback in ~30 seconds, leveraging:

- Context Engineering
- Spec-Driven Development
- Multi-Agent Reviews
- Adversarial LLM Review
- Continuous Code Quality Loop



AI as Review Workflow

(not AI as Reviewer)

Instead of a standalone AI Code Reviewer on top of PRs, **CloudFlare** built a CI-Native AI-powered review system.

In 1 month, it ran 130,000+ reviews, averaging 1.2 findings per review:

- Multi-Agent Reviews
- Shift Left
- Deterministic Engineering Practices
- Continuous Code Quality Loop



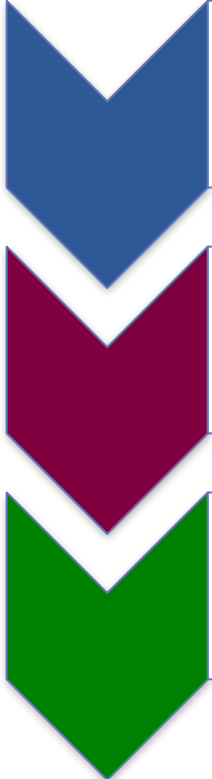
Code is Cheap; Attention is Not

From early to mid-2025, 90% of **Block**'s 3500 engineers were using AI daily. Even with AI Code Reviews, many PRs sat untouched.

How they eliminated the bottleneck:

- Small Incremental Changes
- Shift Left
- Adversarial LLM Review
- Multi-Agent Reviews
- Continuous Code Quality
- Context Engineering

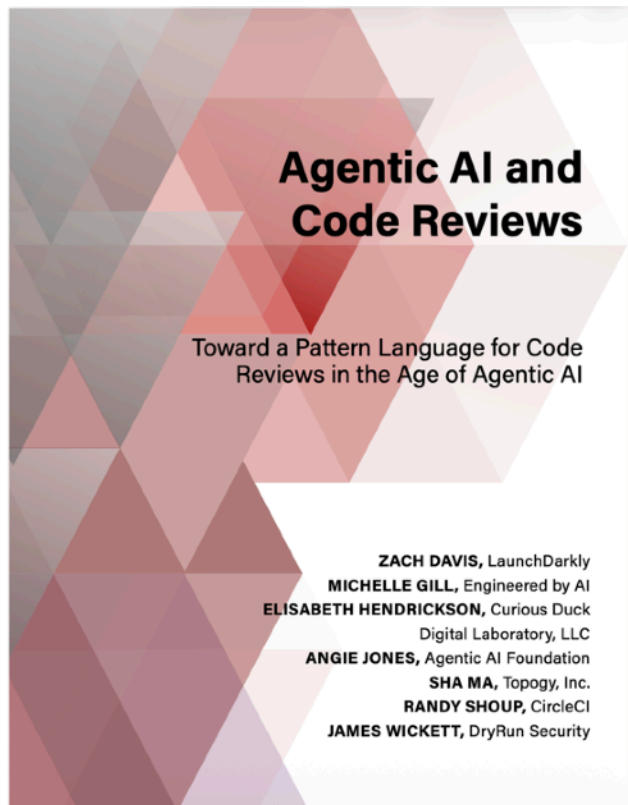
AI and Code Reviews

- 
- Code Review as Constraint
 - Patterns of Review
 - Case Studies

It is impossible to produce code at machine speed,
and continue to evaluate code at human speed.

Thank you!

... Co-authors and IT Revolution



Call for Help

- Evolving / congealing practices?
- Counterexamples and tradeoffs?
- Additional case studies?